

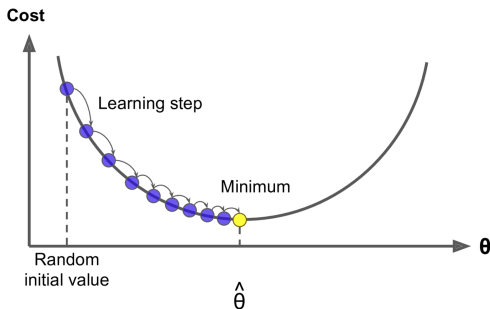
CSE 256 | Lecture 5: FeedForward Neural Networks

Ndapa Nakashole

January 20, 2026

Recap: Optimization with Gradient Descent

- ▶ **Want:** parameters that minimize the loss function $L(\mathbf{W})$
- ▶ **Gradient descent:** start from an initial guess of $\mathbf{W}$
 - ◉ Compute the gradient $\nabla_{\mathbf{W}} L(\mathbf{W})$
 - ◉ Take a small step in the direction of **steepest descent**
 - ◉ Repeat until convergence



Recap: Analytic Gradients of the Softmax Loss

Gradient for the target class

$$\frac{\partial L_i(W)}{\partial \mathbf{W}_{y_i}} = -x_i + p(y_i \mid x_i; W) x_i$$

Gradient for non-target classes

$$\frac{\partial L_i(W)}{\partial \mathbf{W}_{y'}} = p(y' \mid x_i; W) x_i$$

Gradient descent makes each class weight vector look more like the training examples of that class.

Today

- ① A Single Neuron - extending linear models with nonlinearity
- ② Combining Neurons into Feedforward Neural Networks (FFNs)
- ③ How Neural Networks are Trained
- ④ A Simple FNN for NLP: Deep Averaging Networks (PA1)

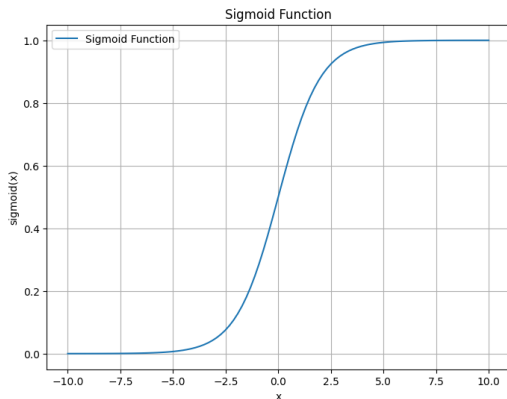
Single Neuron

From Multi-Class to Binary Classification

Simpler Case of Two Classes

- ▶ Softmax models probabilities over **many classes**
- ▶ In case only two classes, one probability determines the other
 - ◉ Given $p(y = 1|x)$, we can compute $p(y = 0|x)$
 - ◉ only need **single score** and the sigmoid function to squash into into probability

Sigmoid Function



- ▶ Squashes any real number into $[0, 1]$
- ▶ Large positive score $\rightarrow$ high confidence in class 1
- ▶ Large negative score $\rightarrow$ high confidence in class 0

Binary Linear Classifier: Sigmoid vs. Softmax

Multi-class (Softmax):

► **Scores (vector):** $\mathbf{s} = \mathbf{W}x_i$

► **Probabilities:**

$$P(Y = k) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Binary (Sigmoid):

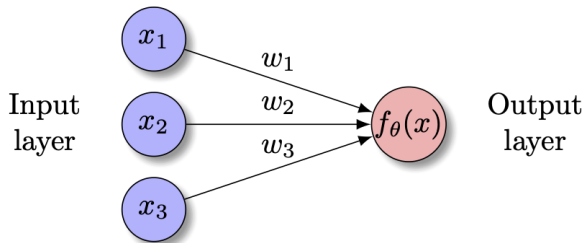
► **Score (scalar):** $s = \mathbf{w}^\top x_i$

► **Probability:**

$$P(Y = 1) = \sigma(s) = \frac{1}{1 + e^{-s}}$$

► $P(Y = 0) = 1 - P(Y = 1)$

Binary Linear Classifier = One Neuron



- ▶ **Inputs:** feature vector $x = (x_1, x_2, x_3)$
- ▶ **Weights:** $w = (w_1, w_2, w_3)$
- ▶ **Score (pre-activation):**

$$f_{\theta}(x) = \mathbf{w}^{\top} x$$

We can expand $f_{\theta}(x)$ as follows:

$$f_{\theta}(x) = w_1 \cdot x_1 + w_2 \cdot x_2 + w_3 \cdot x_3$$

A Neuron as a Binary Logistic Regression Unit

A neuron is defined by:

- ▶ **Weight vector:** $\mathbf{w} \in \mathbb{R}^d$
- ▶ **Activation function:** $g : \mathbb{R} \rightarrow \mathbb{R}$

Given an input vector $x \in \mathbb{R}^d$, the neuron computes:

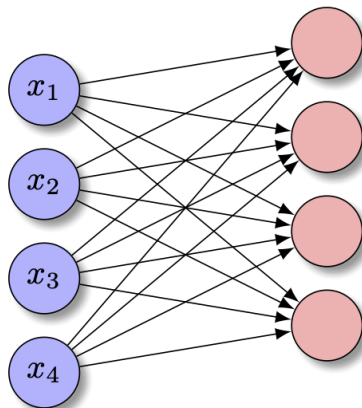
$$h = g(\mathbf{w}^\top x)$$

Activation functions can be (ReLU, tanh, etc) instead of sigmoid.

Network of Neurons

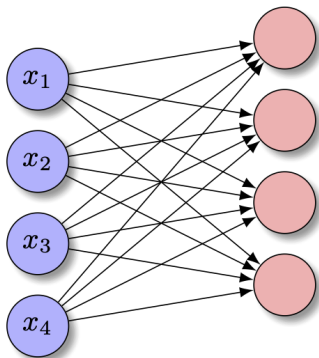
A Neural Network = Many Logistic Regressions in Parallel

- ▶ Each neuron computes a logistic regression-style output
- ▶ If we apply many neurons to the same input, we obtain a **vector of outputs**
- ▶ This vector forms the output of a **layer**



A Neural Network = Many Logistic Regressions in Parallel

If we feed a vector of inputs through a bunch of logistic regression functions, then we get a vector of outputs ...

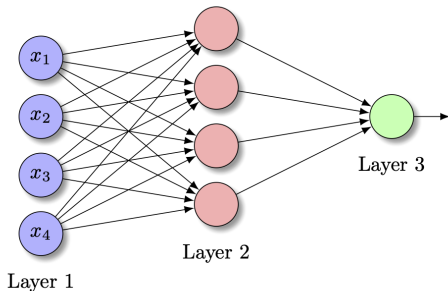


Layer 1

But we don't have to decide ahead of time what variables these logistic regressions are trying to predict!

A Neural Network = Composing Logistic Regressions

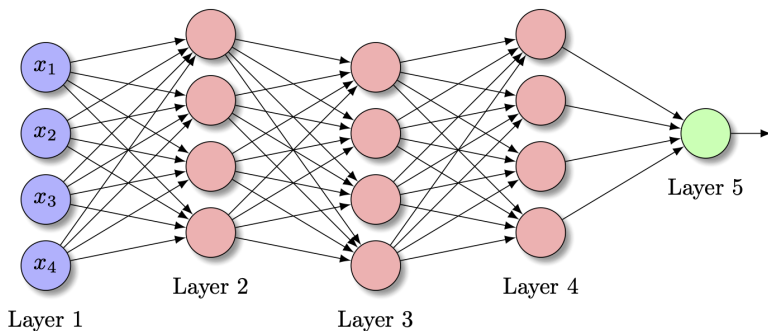
- ▶ A layer maps an input vector to an output vector
- ▶ That output can be fed into another logistic regression layer



- ▶ The outputs of intermediate layers are **not predefined**
- ▶ The **loss function** determines what these hidden values should represent in order to predict the final target

A Neural Network = Composing Logistic Regressions

Before we know it, we have a **multi-layer neural network**:



- ▶ Each layer **re-represents** the data
- ▶ The final classifier can be **highly non-linear** with respect to the original input x

Matrix Notation for a Layer

A layer with multiple neurons is associated with a **weight matrix** W , whose rows correspond to individual neurons.

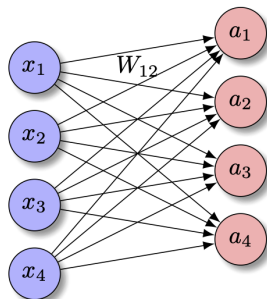
$W \in \mathbb{R}^{m \times d}$ where m is the number of neurons and d is the input dimension.

$$a_1 = g(W_{11}x_1 + W_{12}x_2 + W_{13}x_3 + W_{14}x_4 +$$

$$a_2 = g(W_{21}x_1 + W_{22}x_2 + W_{23}x_3 + W_{24}x_4 +$$

$\vdots$

$$a_i = g\left(\sum_j W_{ij}x_j + b_i\right)$$



Layer 1

- Each neuron computes one activation a_i

Matrix Notation for a Layer

$$z = Wx + b$$

$$a = g(z)$$

- ▶ x : input vector
- ▶ W : weight matrix
- ▶ b : bias vector
- ▶ a : output (activations)

Activation functions are applied
element-wise:

$$g([z_1, z_2, z_3]) = [g(z_1), g(z_2), g(z_3)]$$

Stacking Layers in Matrix Form

A deep neural network is obtained by **repeating the same computation**:

$$\begin{aligned}h^{(1)} &= g\left(W^{(1)}x + b^{(1)}\right) \\h^{(2)} &= g\left(W^{(2)}h^{(1)} + b^{(2)}\right) \\&\vdots \\h^{(L)} &= g\left(W^{(L)}h^{(L-1)} + b^{(L)}\right)\end{aligned}$$

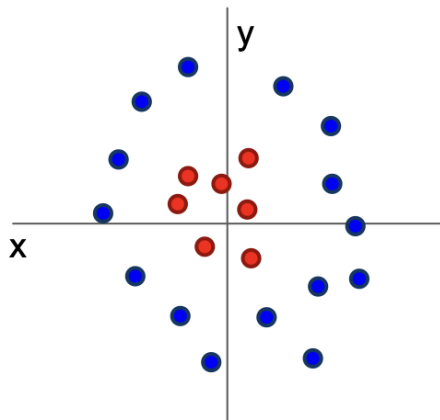
- ▶ Each layer applies a linear map followed by a nonlinearity
- ▶ The output of one layer is the input to the next

Activation Function g , why nonlinearities?

- ▶ The activation function g must be **nonlinear**
- ▶ Without nonlinearity, stacking layers provides no extra power

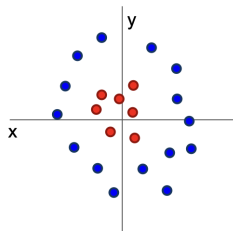
Example:

In the original input space, the data is not linearly separable



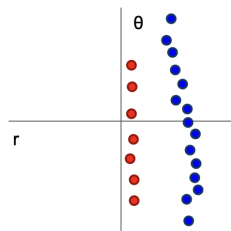
Why Nonlinearities?

After a nonlinear transformation, the data becomes linearly separable



Cannot separate red and blue points with linear classifier

$$f(x, y) = (r(x, y), \theta(x, y))$$

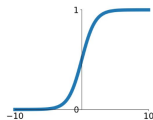


After applying feature transform, points can be separated by linear classifier

Activation functions

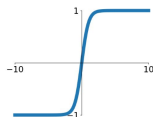
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



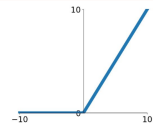
tanh

$$\tanh(x)$$



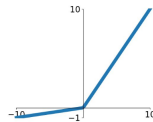
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

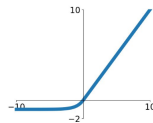


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

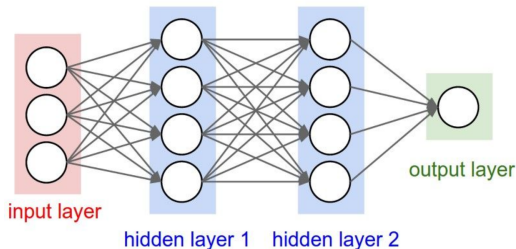
ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



ReLU is a good default choice for most problems

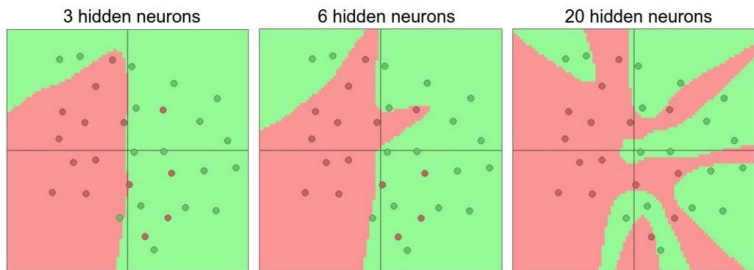
Example feed-forward computation of a neural network



```
# forward-pass of a 3-layer neural network:  
f = lambda x: 1.0 / (1.0 + np.exp(-x)) # activation function (use sigmoid)  
x = np.random.randn(3, 1) # random input vector of three numbers (3 × 1)  
h1 = f(np.dot(W1, x) + b1) # calculate first hidden layer activations (4×1)  
h2 = f(np.dot(W2, h1) + b2) # calculate second hidden layer activations (4×1)  
out = np.dot(W3, h2) + b3 # output neuron (1×1)
```

Setting the number of layers and their sizes

More layers, more neurons $\rightarrow$ more capacity to learn complex functions



↑
more neurons = more capacity

Plugging a Neural Network into a Loss Function

Neural network score function (2 layers):

$$s = f(x; W_1, W_2) = W_2 g(W_1 x) \quad (\text{nonlinear score function})$$

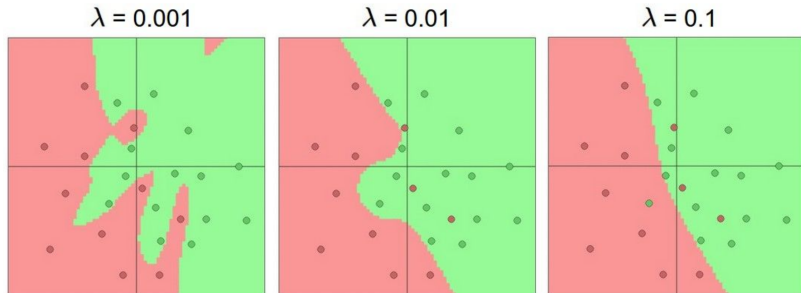
Loss on predictions (softmax loss):

$$L_i = -\log \left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}} \right) \quad (\text{measures prediction error})$$

Total training objective:

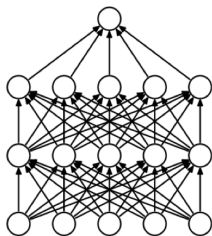
$$L = \frac{1}{N} \sum_{i=1}^N L_i + \lambda R(W_1) + \lambda R(W_2) \quad (\text{data loss} + \text{regularization})$$

Why Regularization Matters in Neural Networks

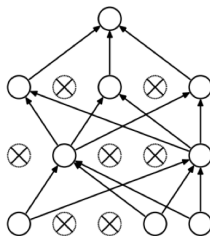


Increasing regularization strength λ trades **model complexity** for **generalization**.

Regularization in Neural Networks: Dropout



(a) Standard Neural Net



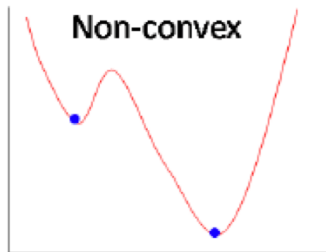
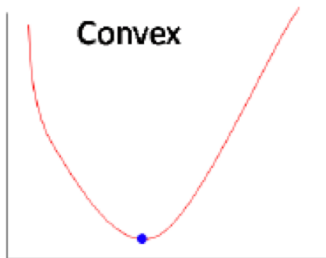
(b) After applying dropout.

- ▶ **Training time:** randomly set some activations to zero
- ▶ **Inference time:** use the full network (no dropout)
- ▶ **Intuition:** Encourage redundancy and robustness
- ▶ **Implement:** with one line in PyTorch

Parameter Estimation in Neural Networks: Gradient Descent

Try multiple and average the results ...

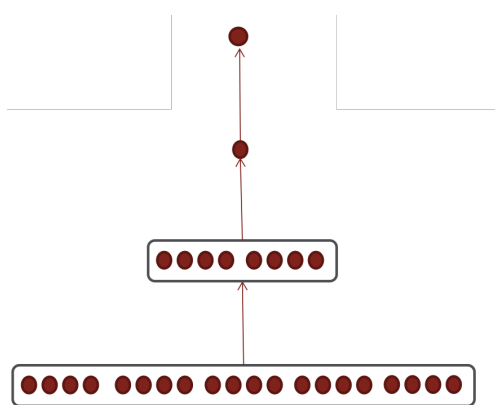
- ▶ Due to non-linearities, the neural network functions are not convex, and **gradient descent can get stuck in local minima**



In practice gradient descent works well on big complex neural models, why? there are **many winning tickets**. see *"The lottery Ticket hypothesis"*, ICLR 2019

Text Classification with FFNs

Convert text to vector, apply FFN, pick label with highest score



$$p = \text{softmax}(s)$$

$$s = \mathbf{W}_2 \mathbf{h}$$

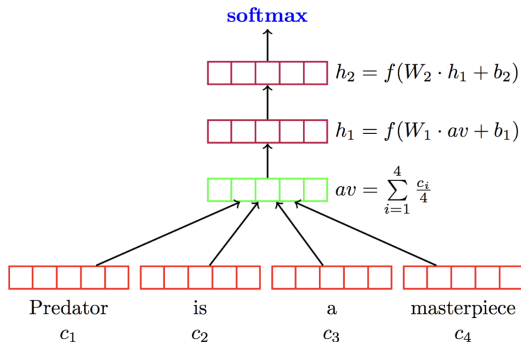
$$\mathbf{h} = g(\mathbf{W}_1 \mathbf{x} + \mathbf{b})$$

$\mathbf{x}$ (input)

FFN Architectures in NLP, example: **Deep
Averaging Networks**

FFN Architectures in NLP: Deep Averaging Networks (PA1)

- ▶ Deep Averaging Networks: FNN is fed the average of word embeddings from input text



- ▶ Widely-held view: need to model syntactic structure to represent language
- ▶ DAN: Surprising that averaging can work as well as it does

Deep Averaging Networks

Sentiment Analysis

		Model	RT	SST fine	SST bin	IMDB	Time (s)	
Bag-of-words		DAN-ROOT	—	46.9	85.7	—	31	Iyyer et al. (2015)
		DAN-RAND	77.3	45.4	83.2	88.8	136	
		DAN	80.3	47.7	86.3	89.4	136	
	{	NBOW-RAND	76.2	42.3	81.4	88.9	91	Wang and Manning (2012)
		NBOW	79.0	43.6	83.6	89.0	91	
		BiNB	—	41.9	83.1	—	—	
		NBSVM-bi	79.4	—	—	91.2	—	
	{	RecNN*	77.7	43.2	82.4	—	—	Kim (2014)
		RecNTN*	—	45.7	85.4	—	—	
		DRecNN	—	49.8	86.6	—	431	
		TreeLSTM	—	50.6	86.9	—	—	
		DCNN*	—	48.5	86.9	89.4	—	
		PVEC*	—	48.7	87.8	92.6	—	
		CNN-MC	81.1	47.4	88.1	—	2,452	
		WRRBM*	—	—	—	89.2	—	

Training Neural Networks

Hyperparameters of a Feedforward Neural Network

► Architecture

- ◉ Number of layers (depth)
- ◉ Size of each layer (width)

► Nonlinearity

- ◉ Activation function (ReLU, Sigmoid, ...)

► Regularization

- ◉ L2 weight decay, Dropout, ...

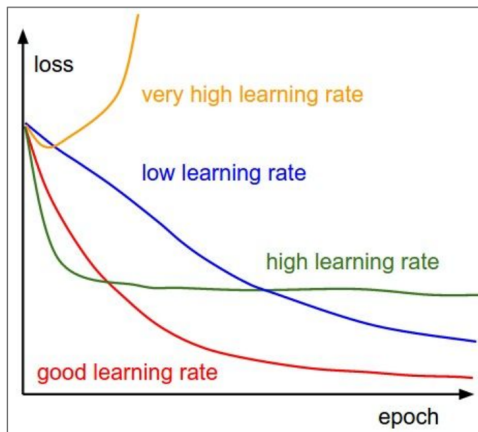
► Optimization

- ◉ Learning rate (step size in gradient descent)

► ...

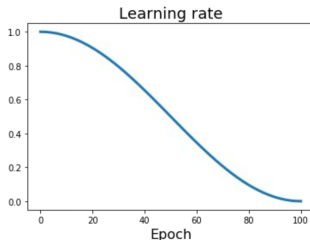
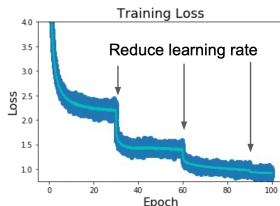
Learning Rate is a crucial Hyperparameter

- ▶ **Learning rate** (η): step size in gradient descent
- ▶ Too small $\rightarrow$ very slow training
- ▶ Too large $\rightarrow$ unstable updates or divergence



Learning Rate Schedules: Change the Learning Rate Over Time

- ▶ Start with a larger learning rate to make fast progress
- ▶ Gradually reduce it to refine the solution



- ▶ **Step schedule:**
 - ▶ Reduce learning rate at a few fixed epochs (common)
- ▶ **Cosine schedule:**
 - ▶ Smoothly decreases the learning rate over time

Putting It All Together: How Neural Networks Learn

Architecture	Optimization	Regularization
▶ Number of layers ▶ Layer widths ▶ Activation functions	▶ Gradient descent ▶ Learning rate ▶ Learning-rate schedules	▶ L2 weight decay ▶ Dropout ▶ Early stopping

Architecture determines *what can be learned*

Optimization determines *how it is learned*

Regularization determines *which solutions are preferred*

Training notes

- ▶ Feedforward neural networks can be implemented easily in PyTorch
 - ◉ Different activation functions available: ReLU, Sigmoid, etc.
 - ◉ Different regularization techniques available: Dropout, L2, etc.
 - ◉ Different learning rate schedules available: Step, Cosine, etc.
 - ◉ Different optimizers available: SGD, Adam, etc.
 - ◉ Different loss functions available: CrossEntropy, MSELoss, etc.
- ▶ For PA1, as with a linear model, most minor tweaks like dropout, etc. will make $< 1\%$ difference. If you're 10% off the performance target, it's likely due to a mis-sized network, poor optimization, bugs, etc.

Where can I find text classification data: NLP benchmarks



Hugging Face Search models, datasets, users...

Models Datasets Spaces Docs Solutions Pricing Log In Sign Up

Task Categories

- text-classification conditional-text-generation
- sequence-modeling question-answering structure-prediction
- other + 37

Tasks

- machine-translation language-modeling
- named-entity-recognition extractive-qa
- sentiment-classification summarization + 215

Languages

- en es de fr pt it + 187

Multilinguality

- monolingual multilingual translation
- other-programming-languages fa en + 4

Sizes

- 10K<n<100K 1K<n<10K 100K<n<1M 1M<n<10M unknown
- n<1K + 15

Datasets 2,630 Search Datasets T1 Sort: Most Downloads

glue Updated Apr 14, 2021 · + 593k · ♡ 15	common_voice Updated Dec 6, 2021 · + 269k · ♡ 20
super_glue Updated Nov 22, 2021 · + 170k · ♡ 6	blimp Updated Nov 22, 2021 · + 145k
imdb Updated Nov 5, 2021 · + 111k · ♡ 3	wikitext Updated Nov 9, 2021 · + 94.3k · ♡ 5
squad Updated Jul 19, 2021 · + 76.6k · ♡ 14	wmt16 Updated Nov 30, 2020 · + 48.7k · ♡ 2
tweets_hate_speech_detection Updated Dec 6, 2021 · + 45.8k · ♡ 1	squad_v2 Updated Jul 7, 2021 · + 43.4k · ♡ 2
anli Updated Apr 14, 2021 · + 42.4k · ♡ 3	librispeech_asr Updated Dec 6, 2021 · + 42.1k · ♡ 6
trec Updated Apr 26, 2021 · + 37.7k · ♡ 2	xnli Updated Nov 22, 2021 · + 30.6k · ♡ 5

See: <https://huggingface.co/datasets>

Text Classification: summary and next steps

- ① The Text Classification Problem
- ② Example Applications
- ③ Models (Logistic Regression, Feedforward Neural Networks)

Next Time: Word Representations ...

Questions?